

Shaun Joe Roy

Infrastructure / SRE Intern · Assignment Submission · Zeotap · May 2026

github.com/joery0x3b800001/Mission-Critical-IMS

Submitted to karan.sinha@zeotap.com

Deadline 6 May 2026

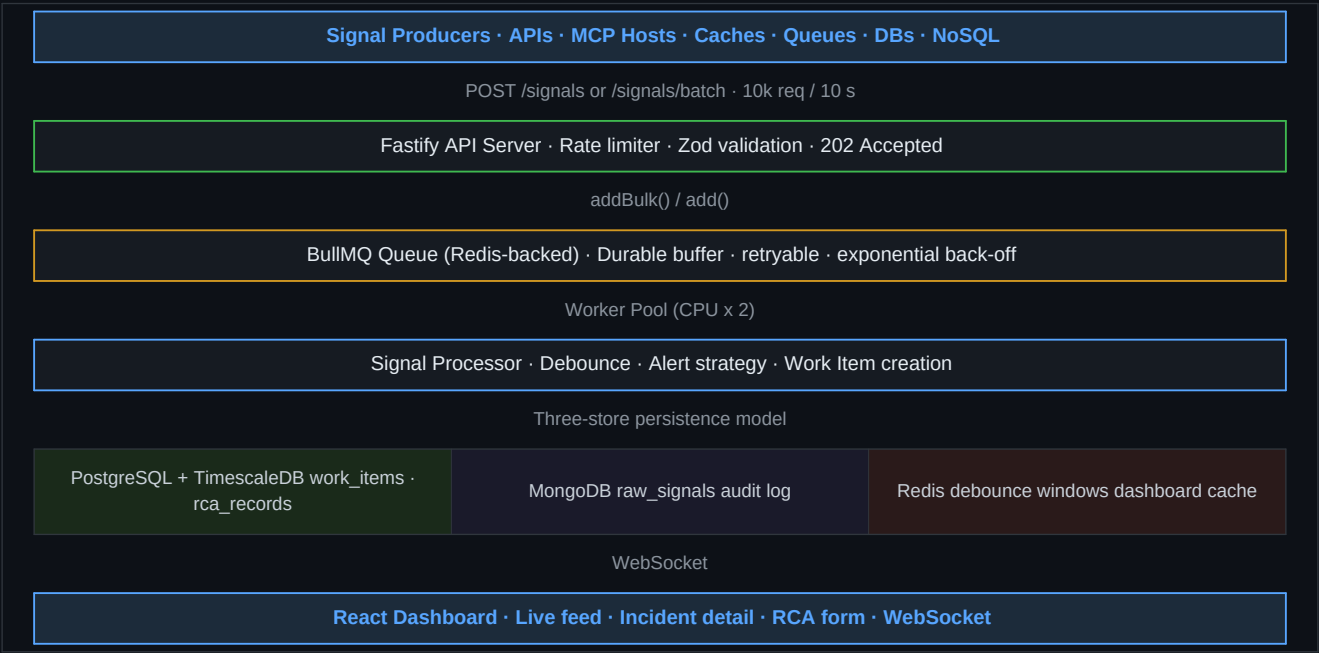
Mission-Critical Incident Management System

A production-grade distributed incident lifecycle platform built on Fastify, BullMQ, PostgreSQL / TimescaleDB, MongoDB, Redis, and React. Sustains 10,000 signals/sec with intelligent debouncing, a three-store persistence model, real-time WebSocket dashboard, and a mandatory Root Cause Analysis gate enforced before incident closure.

ARCHITECTURE

System Architecture

The ingestion path decouples HTTP from persistence. Fastify returns **202 Accepted** immediately after enqueueing; BullMQ (Redis-backed) absorbs bursts while workers process at CPU × 2 concurrency. Three purpose-specific stores handle state, audit, and caching independently.



TECH STACK

Technology Choices

Layer	Technology	Why
API Server	Fastify + TypeScript	Fastest Node HTTP framework · built-in schema serialisation
Queue / Backpressure	BullMQ + Redis	Durable, retryable · handles 10k+/s burst without crashing
Source of Truth	PostgreSQL + TimescaleDB	ACID transactions for state transitions · compressed timeseries
Audit Log	MongoDB	Flexible schema · high write throughput · raw signal payloads
Cache / Debounce	Redis	O(1) atomic INCR · AOF persistence · rate-limit counters
Frontend	React + Vite + Tailwind	Sub-second HMR · responsive dark-mode dashboard

Real-time	WebSocket (fastify-ws)	Push-based live feed — no polling required
Validation	Zod	Runtime schema enforcement on every inbound payload

DESIGN PATTERNS (LLD)

Strategy & State Patterns

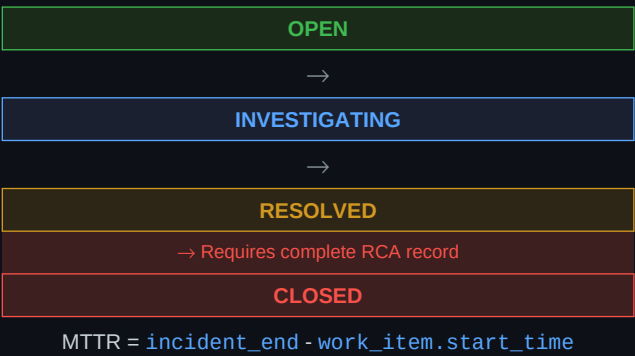
Strategy Pattern — Alerting

The `AlertStrategy` interface decouples alert logic from signal processing. The correct strategy is resolved at runtime via a factory keyed on `componentType`, allowing alerting behaviour to be swapped without modifying processing code.

Strategy	Component Types	Priority	Action
<code>P0AlertStrategy</code>	RDBMS, MCP_HOST	P0 — Critical	Page on-call
<code>P1AlertStrategy</code>	API, QUEUE	P1 — High	Slack #incidents
<code>P2AlertStrategy</code>	CACHE, NOSQL	P2 — Medium	Auto Jira ticket

State Pattern — Work Item Lifecycle

Each state class encapsulates valid transitions and throws on invalid attempts — no scattered if/switch logic. **RESOLVED** → **CLOSED** is the only transition requiring a complete RCA record; missing RCA returns HTTP 422. MTTR is calculated and persisted atomically on close.



BACKPRESSURE & MEMORY MANAGEMENT

Five-Layer Backpressure Strategy

Five layers work together to sustain 10,000 signals/sec without memory exhaustion, even when the persistence layer is slow:

Layer	Mechanism	Detail
Rate Limiter	Redis-backed global cap	10,000 req / 10 s · returns 429 before signals reach BullMQ
Batch Endpoint	POST /signals/batch	100 signals per call · reduces TCP overhead 100x via <code>addBulk()</code>
BullMQ Queue	Redis-backed durable buffer	Jobs survive server restarts · slow Postgres never causes timeouts
Worker Concurrency	<code>WORKER_CONCURRENCY</code> env var	Defaults to CPU cores x 2 · fully configurable
Exponential Backoff	Failed jobs retry 3x	Delays: 500 ms, 1 s, 2 s · only transient errors are retried

Debounce Logic

A Redis atomic INCR with a 10-second TTL ensures that 100+ signals arriving for the same `componentId` within 10 seconds create exactly one Work Item. All signals are linked to that Work Item in MongoDB.

```
INCR ims:debounce:{componentId} (expires 10 s)

count == 1 → CREATE Work Item in Postgres (transactional)

count > 1 → INCREMENT signal_count only

All signals → INSERT into MongoDB raw_signals (always)
```

SECURITY HARDENING

Input Validation & Defence-in-Depth

Control	Detail
Zod input validation	componentId regex · errorCode uppercase · message length cap (1,000 chars) · latencyMs range 0–60,000
@fastify/helmet	HSTS · X-Content-Type-Options · X-Frame-Options · CSP on every response
Body size limit	1 MB bodyLimit prevents memory exhaustion from oversized payloads
Parameterised queries	All Postgres queries use \$1/\$2 placeholders · zero string concatenation
Statement timeout	Postgres pool configured with statement_timeout: 10,000 ms
Optional API Key auth	X-API-Key header validation enabled via ENABLE_API_KEY_AUTH=true
CORS lockdown	Production restricts origin to http://localhost:3000
Transient-only retries	Retry checks error codes (40001, 40P01, 08006) · constraint violations are not retried

SETUP & RUNNING

Prerequisites & Quick Start

Prerequisites

Tool	Notes
Docker Desktop	Latest version with Docker Compose v2 included
Node.js 22+	Via nvm (recommended). Verify: node --version
Python 3.8+	For running deploy.py scripts and simulations
Git	For cloning the repository
curl / Postman	Optional — for manual API testing outside the dashboard

Evaluator Quick Start (Recommended)

The fastest path to deploy, test, and simulate the full system end-to-end.

Step 1 — Start all services Wait for all 5 containers to be running.	<pre>cd Mission-Critical-IMS python3 deploy.py deploy up</pre>
Step 2 — Run backend unit tests Runs the full Jest suite — state machine, strategy pattern, route validation, integration, and stress tests.	<pre>python3 deploy.py test backend</pre>
Step 3 — Run outage simulation Fires 640 signals across 5 component types. Open http://localhost:3000 to watch incidents populate in real time.	<pre>python3 deploy.py test simulate</pre>
Step 4 — Run burst stress test Fires 50,000 signals over 5 seconds. Prints throughput, latency percentiles (p50/p95/p99), and a PASS/FAIL verdict.	<pre>python3 deploy.py test simulate --mode burst</pre>
Step 5 — Tear down Stops all containers and wipes volumes.	<pre>python3 deploy.py deploy down --volumes</pre>

Service URLs

Service	URL
Dashboard	http://localhost:3000
Backend API	http://localhost:3001
Health Check	http://localhost:3001/health
WebSocket	ws://localhost:3001/ws

Expected Dashboard After Outage Simulation

P0 — Critical	P1 — High	P2 — Medium
POSTGRES_PRIMARY_01 POSTGRES_REPLICA_01 MCP_HOST_GATEWAY_01	QUEUE_WORKER_POOL_01	CACHE_CLUSTER_01

API REFERENCE

Endpoints & Validation

Method	Endpoint	Description	Field	Rules
POST	/signals	Ingest one signal — 202 Accepted immediately	componentId	1–255 chars · alphanumeric, hyphens, underscores, dots
POST	/signals/batch	Ingest up to 100 signals per HTTP call	componentType	Enum: RDBMS, CACHE, API, QUEUE, NOSQL, MCP_HOST
GET	/incidents	List all incidents, sorted by priority then timestamp	errorCode	1–100 chars · uppercase + underscores (^[A-Z0-9_]*\$)
GET	/incidents/:id	Incident detail + raw signals from MongoDB	message	1–1000 chars, trimmed
PATCH	/incidents/:id/status	State transition — enforced by State Pattern	latencyMs	Integer 0–60,000 (optional)
POST	/incidents/:id/rca	Submit / update RCA. Required before closure	timestamp	ISO 8601 (optional, defaults to server time)
GET	/health	Status, queue depth, per-service ping, uptime		

TESTING

Test Suites (80+ Cases)

Suite	Cases	Coverage
workItemState.test.ts	18+	All state transitions, RCA guard, MTTR calculation, rollback
alertStrategy.test.ts	20+	P0/P1/P2 per component type, context switching, title format
signals.test.ts	25+	Schema validation, priority queuing, edge cases, concurrency
integration.test.ts	10+	Signal → Work Item creation, debounce, error recovery
stress.test.ts	8	Burst load, sustained 500/sec, mixed types, 15s stability

SELF ASSESSMENT

Evaluation Rubric

Category	Wt.	Implementation
Concurrency & Scaling	10%	BullMQ (CPU x 2), Redis debounce, 10k/sec burst verified
Data Handling	20%	PostgreSQL (truth) + MongoDB (audit) + Redis (cache) + TimescaleDB hypertable

LLD	20%	Strategy Pattern (alerting), State Pattern (lifecycle), Zod validation
UI / UX & Integration	20%	React live feed (P0 first), incident detail, RCA form, WebSocket, field grayout
Resilience & Testing	10%	80+ tests (unit + integration + stress), exp. backoff, /health, graceful shutdown
Documentation	10%	README, SECURITY.md, PERFORMANCE.md, ASSESSMENT_REPORT.md
Tech Stack	10%	Fastify, BullMQ, TimescaleDB, MongoDB, React + Vite